

Skill-Based Barber Role Predictor

AI Course Project Documentation

1. Introduction

In modern barbershops it is often difficult to decide what professional level a barber has. Usually managers just look at years of experience, but this is not always correct. A barber with 10 years of experience who only does basic haircuts is not the same as a barber with 5 years who knows coloring, chemical perm and hair extensions.

The goal of this project is to build a machine learning system that can predict the professional level of a barber based on his skills, experience and education. The system outputs one of four levels: Junior, Middle (Barber), Senior, or Top (Master Barber), together with an estimated salary range in KZT for the Kazakhstan market.

This project is integrated into a real barbershop booking web application (BarberHub) built with FastAPI and Next.js.

2. Dataset

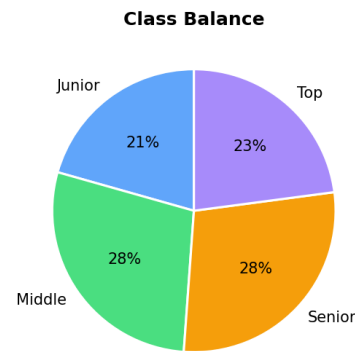
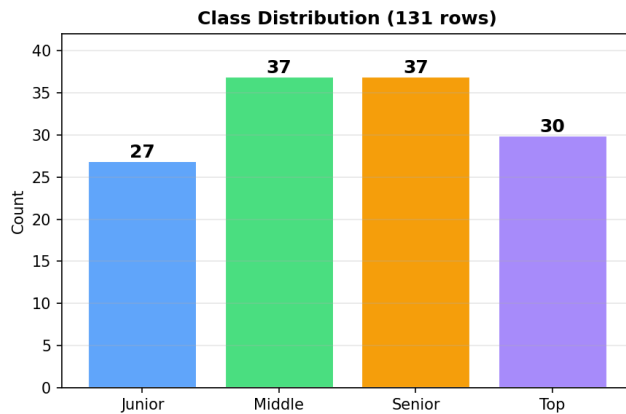
Since there is no public dataset for barber skill levels, I created a synthetic dataset based on real market research and barbershop hiring practices in Kazakhstan (Almaty, Astana — hh.kz, 2024–2026).

Dataset size: 131 rows

- 100 balanced examples (25 per class)
- 31 borderline edge cases (stagnant barbers, trending skills, boundary scenarios)

Class distribution:

Class	Count	Description
Junior	27	0 years exp, basic skills
Middle	37	1–5 years, intermediate
Senior	37	3–10 years, advanced + specialist
Top	30	5–10+ years, full specialist



Skills in dataset (16 total):

- **Foundation (×1):** Classic Haircut, Clipper Cut
- **Advanced (×3):** Fade, Beard Sculpting, Straight Razor, Long Haircut (Scissors), HairTattoo, Waxing, Face Treatment
- **Specialist (×5):** Hair Coloring, Color Correction, Hair Extensions, Hair Camouflage, Chemical Perm
- **Client Services (×2):** Style Consulting, Product Knowledge

Important design decision: avg_rating and completed_bookings columns exist in the CSV but are not used as features, because these values are not available when evaluating a new or prospective employee. Only information available at hiring time is used.

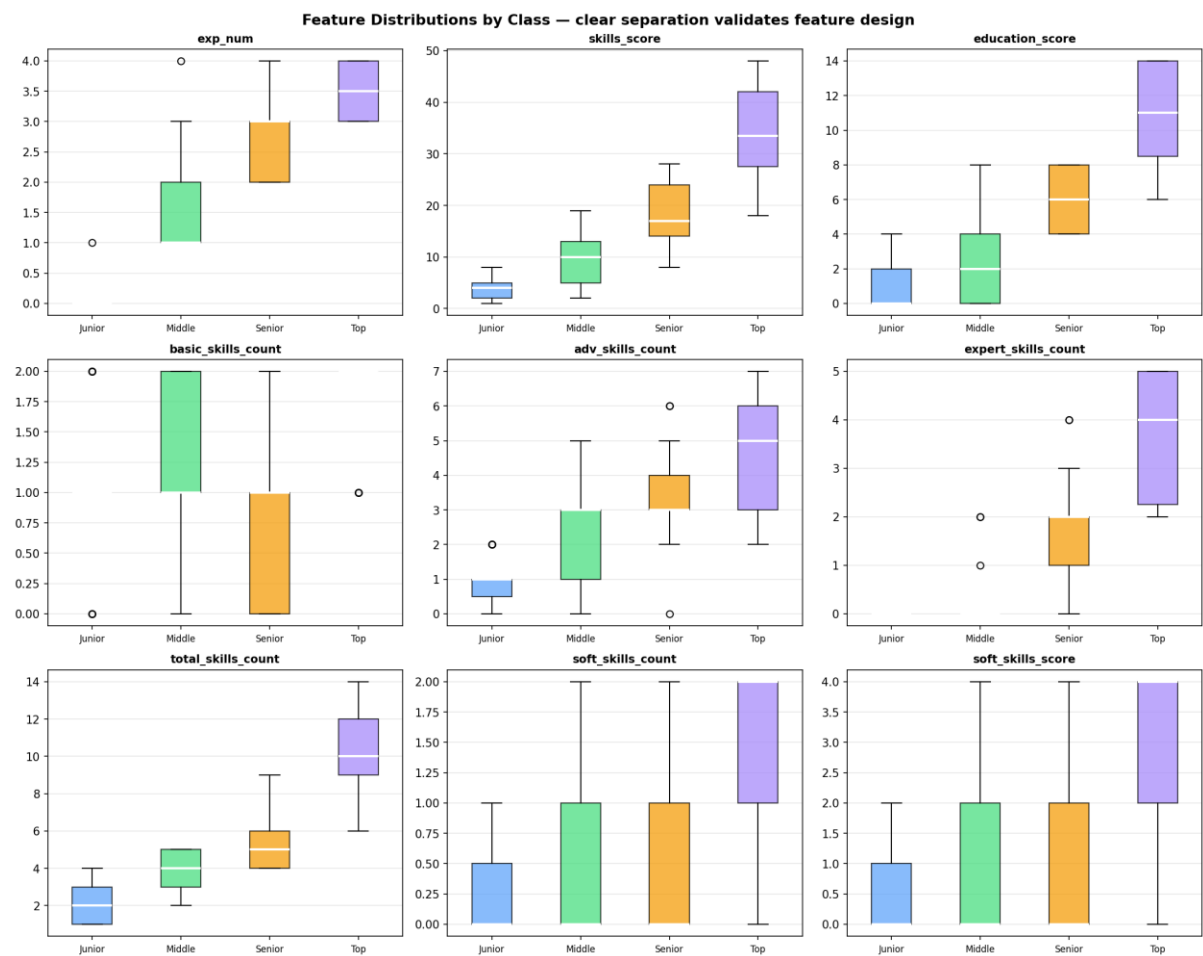
3. Feature Engineering

Raw data (skill list as text, experience as category) cannot be fed directly into a classifier. I applied the following transformations:

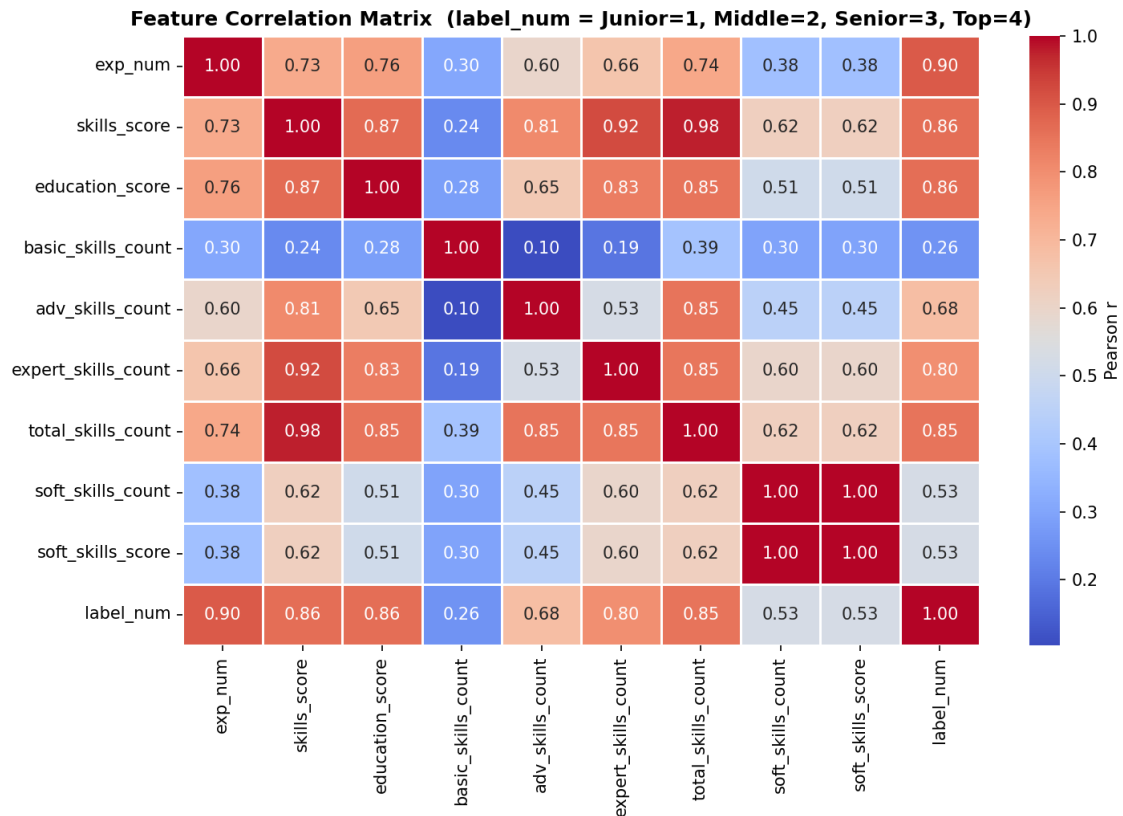
Feature	How it is computed	Rationale
exp_num	Map categories to numbers: 0→0, 1-3→1, 3-5→2, 5-10→3, 10+→4	Ordinal encoding for experience
skills_score	Sum: basic×1 + advanced×3 + specialist×5	Higher weights for rarer, harder skills
education_score	education_count × 2	Linear, more courses = more knowledge
basic_skills_count	Count of foundation skills selected	Separates unqualified from qualified

adv_skills_count	Count of advanced skills	Key signal for Middle vs Senior
expert_skills_count	Count of specialist skills	Key signal for Senior vs Top
total_skills_count	basic + advanced + specialist	Total technical breadth
soft_skills_count	Count of client service skills	Consulting and product knowledge
soft_skills_score	$\text{soft_skills_count} \times 2$	Separate soft skill contribution

Total: 9 features. All features are numerical, which works well with Random Forest.



The box plots show clear separation between classes, especially in `skills_score`, `exp_num`, and `expert_skills_count`. This confirms that the chosen features are informative.



4. AI Technique — Random Forest Classifier

4.1 Algorithm Choice

I chose RandomForestClassifier from scikit-learn as the main algorithm. The reasons are:

1. **Non-linear boundaries** — the relationship between skills and level is not linear. For example, having perm skill matters a lot for Top but having waxing alone does not guarantee Senior.
2. **Handles correlated features** — many skill count features are correlated (e.g., total_skills_count correlates with adv_skills_count). Random Forest handles this better than logistic regression.
3. **Feature importance** — gives interpretable output that shows which features matter most.
4. **Small dataset performance** — bagging reduces overfitting on our 131-row dataset.

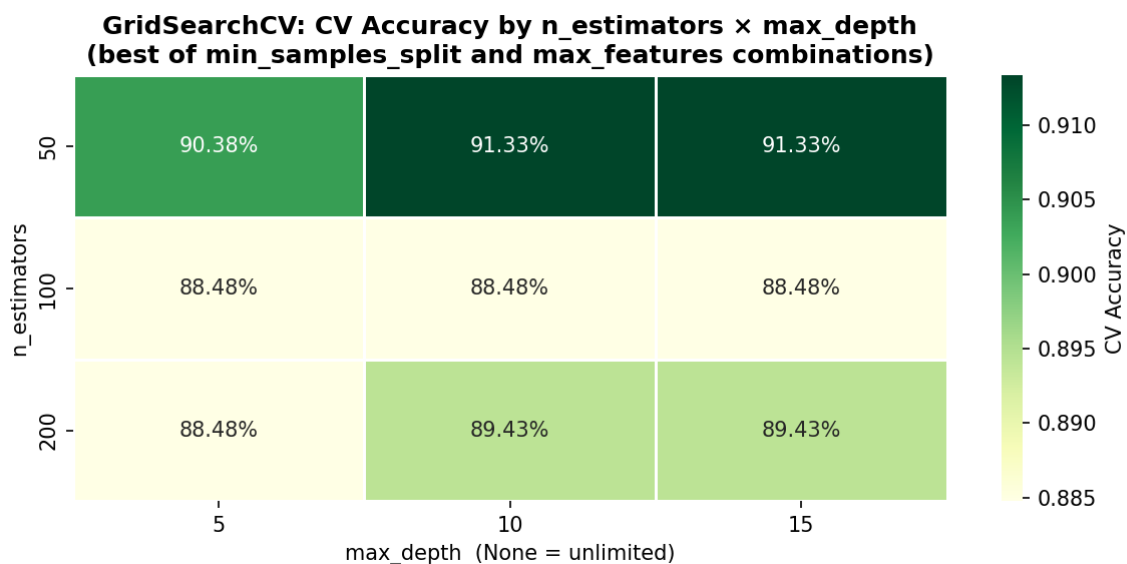
Baseline for comparison: DecisionTreeClassifier (max_depth=5) — simpler, single tree, more likely to overfit.

4.2 Hyperparameter Tuning — GridSearchCV

I used GridSearchCV with 5-fold cross-validation to find the best hyperparameters. This is an exhaustive search over all parameter combinations.

Search space (72 combinations × 5 folds = 360 model fits):

Parameter	Values	Purpose
n_estimators	50, 100, 200	Number of trees — more trees = lower variance
max_depth	None, 5, 10, 15	Controls tree depth — prevents overfitting
min_samples_split	2, 5, 10	Minimum samples needed to split a node
max_features	sqrt, log2	Feature subsampling per split



The heatmap shows how CV accuracy changes across parameter combinations.

5. Results and Evaluation

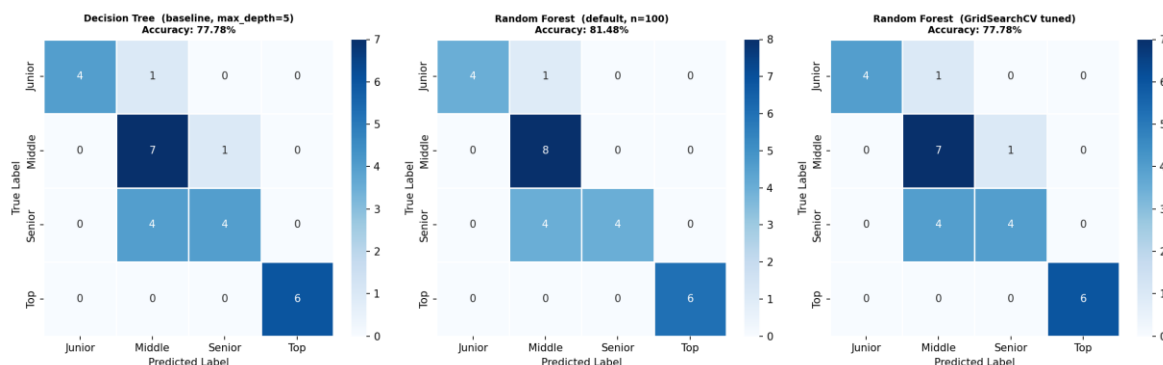
5.1 Model Comparison

Model	Test Accuracy	Weighted F1
Decision Tree (baseline)	77.78%	77.66%

Random Forest (default)	81.48%	81.01%
Random Forest (GridSearchCV tuned)	77.78%	77.66%

The test set contains only 27 samples, so single-split accuracy has high variance.

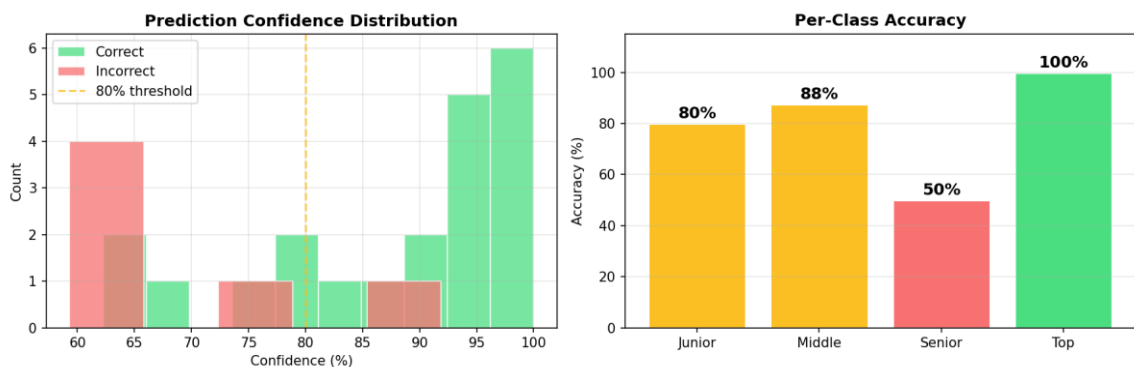
Cross-validation (5-fold, all 131 rows) gives a more reliable estimate: RF Tuned 89.32%, RF Default 90.88%, Decision Tree 84.73%. GridSearchCV best CV score was 91.33%.



5.2 Error Analysis

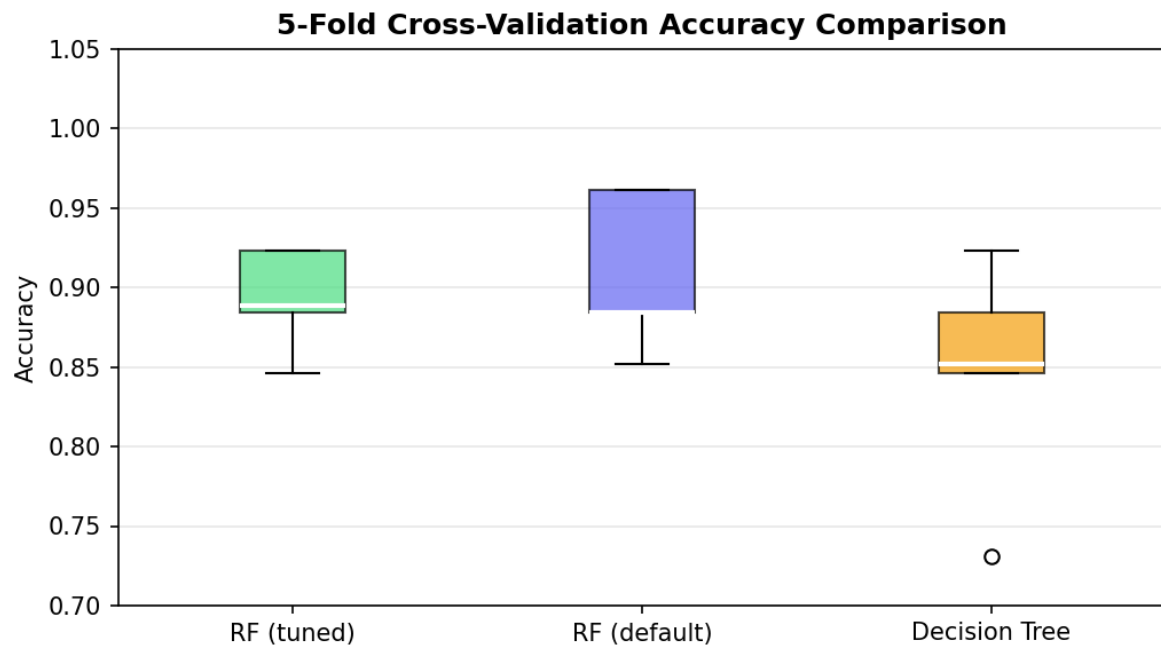
Most misclassifications happen at the Middle/Senior boundary, which is the hardest to separate because:

- Some experienced Middle barbers (3–5 years, few skills) look similar to weak Seniors
- The model is trained on a small dataset so borderline cases are difficult



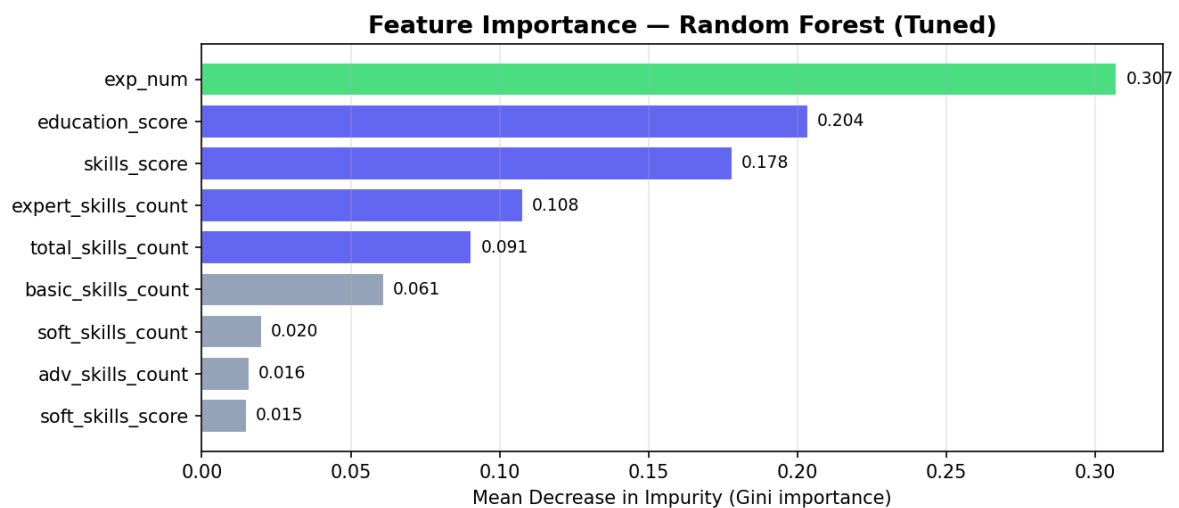
The confidence distribution chart shows that correct predictions have significantly higher confidence than incorrect ones, which means the model "knows when it is uncertain."

5.3 Cross-Validation



Both Random Forest models consistently outperform the Decision Tree across all data splits. RF Tuned achieves 89.32% and RF Default 90.88% mean CV accuracy, compared to 84.73% for the Decision Tree.

6. Feature Importance



The most important features are:

1. **exp_num** — years of experience
2. **education_score** - number of educations
3. **skills_score** — the weighted technical skill total
4. **expert_skills_count** — number of specialist skills (strongest signal for Top level)
5. **total_skills_count** — total skills count

Soft skills (**soft_skills_count**, **soft_skills_score**) have low importance, which is correct — consulting and product knowledge contribute to the profile but do not determine the level alone.

7. Business Logic (Post-Prediction Rules)

The ML model prediction is enhanced by three business rules applied after inference:

Rule 1 — Foundation Qualifier

If a barber has no foundation skills (Classic Haircut or Clipper Cut), the result is "Not Qualified" regardless of all other skills. Every professional barber must demonstrate basic haircut ability.

Rule 2 — Experience Hard Cap

The model cannot predict a level that requires more experience than the barber has:

- 0 years → maximum Junior
- 1–3 years → maximum Middle
- 3–5 years → maximum Senior
- 5+ years → Top possible

This prevents a barber who claims all skills but has 0 experience from being predicted as Top.

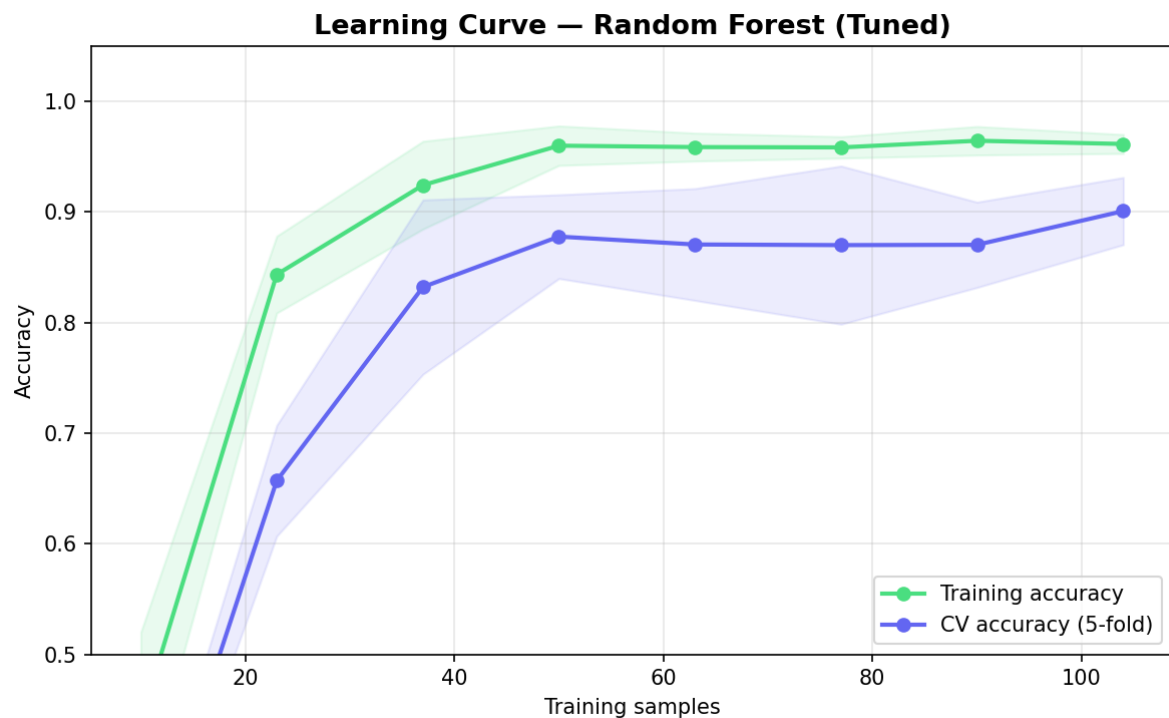
Rule 3 — Skill Floor Check

Minimum skill requirements per level:

- **Senior:** must have at least 2 advanced skills
- **Top:** must have at least 2 advanced AND 2 specialist skills

A barber with 10+ years experience but only classic/machine cuts is correctly assigned Middle, not Senior.

These rules implement domain knowledge that complements the ML model and make the system behave realistically.



8. System Architecture

The project consists of several components working together:

- `ml/generate_dataset.py` — generates the synthetic dataset (131 rows)
- `ml/train.py` — trains the bootstrap model for quick startup
- `ml/barber_grader.ipynb` — full GridSearchCV training with visualizations
- `app/ml/evaluator.py` — loads model, applies business rules, returns prediction
- `app/ml/routes.py` — FastAPI endpoint `/ml/evaluate-barber`
- `frontend/app/admin/ml/page.tsx` — React UI with radar chart, salary display

The system also includes a recency coefficient for education: courses completed recently count fully, while older courses count as 50%. This is implemented on the frontend as a slider.

9. Challenges and Solutions

Challenge 1 — Model predicted Senior just from experience years

Early versions of the model gave too much weight to `exp_num`. A barber selecting "10+ years" with only classic/machine skills was predicted as Senior.

Solution: added stagnant barber examples to training data and implemented the Skill Floor business rule.

Challenge 2 — Expanding skill set while keeping model consistent

When I added 9 new skills (scissors, hair_tattoo, waxing, black_mask, perm, consulting, products), the feature vector changed. I had to update the dataset generation, feature engineering, model training, API endpoint and frontend simultaneously to keep everything in sync.

Challenge 3 — Small dataset size

With only 131 rows, it is easy to overfit.

Solution: I used stratified train/test split, 5-fold cross-validation in GridSearchCV, and borderline edge cases to help the model learn boundary decisions.

10. Conclusion

This project demonstrates a practical ML pipeline for classifying barber professional levels. The system combines a trained RandomForest classifier with explicit business rules to produce realistic, explainable predictions.

Key achievements:

- 9 engineered features from raw skill/experience data
- GridSearchCV over 72 combinations with 5-fold CV
- Three post-prediction business rules for real-world correctness
- Full integration into a working web application
- Kazakhstan-specific salary ranges and trending skill categories (waxing, chemical perm, face treatment)

The model correctly handles edge cases like stagnant barbers (high experience, low skills → Middle), chemical perm alone not qualifying for Top, and soft skills not inflating the level prediction.

Generated dataset, trained model, and all visualizations are reproducible by running `python3 ml/train.py` followed by `ml/barber_grader.ipynb` (Kernel → Restart & Run All).